SESE4143 Software Re-engineering · Semester 7 · Assignment 1 · G1

SREE Assignment 1 G1: Clean Up the Parking Lot

Code Smells, Refactoring & Program Comprehension. Main focus: **identify code smells** and **refactor** legacy Java code without changing its behaviour.

Submission date	Work mode	You need	Outcome
Monday, 26 October 2026	Pairs (2 members)	JDK to run and test your Version 2.0 (recommended)	Spot smells, refactor step by step without changing behaviour, and read code with chunking and cross-referencing

Name	Roll No	Name	Roll No

Course Learning Outcomes assessed

CLO	Course Learning Outcome (from the SESE4143 outline)	Taxonomy	PLO
CLO 1	Explain the concepts and techniques of software reengineering such as models, processes, legacy systems and maintenance procedures.	C2	PLO 2
CLO 3	Analyze maintenance related problems associated with object-oriented software systems including software refactoring and software reuse.	C4	PLO 3

Assignment plan

Section	What you do	CLO	Focus
0	Warm-up: what is a code smell?	CLO 1	Short
A	Spot the smells in Version 1.0	CLO 3	**Core**
B	Refactor step by step (6 recipes) and write Version 2.0	CLO 3	**Core**
C	Match smell-detection tools	CLO 3	Short
D	Benefits check: apply a change request	CLO 3	Short
E	Chunking and cross-referencing	CLO 1	Short
✓	Reflection	CLO 1	Short

The scenario

The university parking office prints parking tickets with the **Version 1.0** program below. It compiles and gives the right output, but nobody wants to touch it. Your job is to re-engineer it into a clean **Version 2.0** that prints *exactly the same output*. Line numbers matter: you will refer to them.

```
 1  class vehicle {
 2      public String n;      // plate number
 3      public int t;         // type: 1 = car, 2 = bike
 4      public int h;         // hours parked
 5      public String z;      // not used anymore
 6
 7      // this method prints the parking ticket
 8      void process() {
 9          int c = 0;
10          if (t == 1) {
11              c = h * 50;
12              if (c > 1000) c = 1000;
13          } else if (t == 2) {
14              c = h * 20;
15              if (c > 1000) c = 1000;
16          }
17          System.out.println("------------------");
18          System.out.println("Plate    : " + n);
19          System.out.println("Hours    : " + h);
20          System.out.println("Charge   : Rs. " + c);
21      }
22
23      void printPass() {
24          System.out.println("------------------");
25          System.out.println("Plate    : " + n);
26          System.out.println("Hours    : " + h);
27          if (t == 1) System.out.println("Type     : Car");
28          else if (t == 2) System.out.println("Type     : Bike");
29      }
30
31      void oldDisplay() {        // nobody calls this
32          System.out.println(n);
33      }
34  }
35
36  public class ParkingSystem {
37      public static void main(String[] args) {
38          vehicle v1 = new vehicle();
39          v1.n = "LEA-1234"; v1.t = 1; v1.h = 6;
40          v1.process();
41          v1.printPass();
42
43          vehicle v2 = new vehicle();
44          v2.n = "LHR-5678"; v2.t = 2; v2.h = 60;
45          v2.process();
46          v2.printPass();
47      }
48  }
```

Expected output (must stay identical after refactoring)

```
------------------
Plate    : LEA-1234
Hours    : 6
Charge   : Rs. 300
------------------
Plate    : LEA-1234
Hours    : 6
Type     : Car
   ... then the same two blocks for LHR-5678
   (60 hours, Charge: Rs. 1000, Type: Bike)
```

0 Warm-up: what is a code smell?

A **code smell** is a surface sign in the code that *usually* points to a deeper design problem. The program still works, so a smell is not a bug. It makes the code harder to read, change and extend, which is why smells are where re-engineering starts.

```
// Smelly
if (age > 18) { ... }
double t = p * 0.17;
```

```
// Clean
if (age > ADULT_AGE) { ... }
double tax = price * TAX_RATE;
```

0.1 Both versions give the same result. In one line, why is the right-hand version better?

0.2 Is a code smell a bug? Circle one and give a reason: **Yes / No**

A Spot the smells in Version 1.0

Read the Version 1.0 code on page 2. Find **at least 8** smells. Use the word bank and write the line number(s) for each one. The first row is done for you.

Word bank (smell names)
Poor / cryptic naming · Naming-convention violation · Meaningless method name · Public fields (no encapsulation) · Magic numbers · Duplicate code · Type code with repeated conditionals (switch-statement smell) · Long method / method doing two jobs · Dead code · Unused field · Comments used as deodorant · No constructor (object built field by field)

#	Smell name	Line(s)	Why does it hurt?
1	*Naming-convention violation*	*1*	*Class names in Java are PascalCase; 'vehicle' looks like a variable*
2			
3			
4			
5			
6			
7			
8			
9			

Hint
Read every name aloud: if you need the comment to know what `n`, `t` or `h` means, that's a smell. Count how many times you see `1000`. Count how many places test `t == 1`. Look for any method that nobody calls. Ask of `process()`: how many different jobs does it do?

B Refactor step by step

Golden rule of refactoring
Refactoring changes the **structure** of code, never its **behaviour**. Make one small change at a time, and after each change the output must still match the expected output on page 2. Run the code after every step.

B1 · Rename (class, field, method, variable)

Fixes: cryptic names, convention violation, meaningless method name

Small example (not the parking code). Before → after:

```
class account {
  double b;      // balance
  void calc() {...}
}
```

```
class Account {
  double balance;
  void addInterest() {...}
}
```

Your task: Rename `vehicle`, `n`, `t`, `h`, `c` and `process()`. Write your new names below (old → new):

Hint
Classes are PascalCase nouns. Methods are verbs that say what they do. If a name makes its comment unnecessary, delete the comment.

B2 · Encapsulate Field + Introduce Constructor

Fixes: public fields, object assembled field by field in `main`

Small example (not the parking code). Before → after:

```
class Point {
  public int x, y;
}
Point p = new Point();
p.x = 3; p.y = 4;
```

```
class Point {
  private final int x, y;
  Point(int x, int y) {
    this.x = x; this.y = y;
  }
}
Point p = new Point(3, 4);
```

Your task: Make the fields `private` and give the class a constructor that takes plate number and hours parked.

Hint
Use `final` for fields that never change after the object is created. Then lines 39 and 44 become one line each.

B3 · Replace Magic Number with Named Constant

Fixes: magic numbers `50`, `20`, `1000`

Small example (not the parking code). Before → after:

```
if (speed > 120) {
  fine = 2000;
}
```

```
static final int SPEED_LIMIT = 120;
static final int SPEEDING_FINE = 2000;
if (speed > SPEED_LIMIT) {
  fine = SPEEDING_FINE;
}
```

Your task: Give each number in the charge rules a name. List your constants below:

Hint
Constants are `static final` and written in UPPER_SNAKE_CASE. Which constant is shared by everyone, and which differ per vehicle type?

B4 · Extract Method

Fixes: duplicate code (lines 17-19 vs 24-26, and the cap on lines 12 and 15), method doing two jobs

Small example (not the parking code). Before → after:

```
void printReceipt() {
  System.out.println("Shop");
  System.out.println(date);
  System.out.println(total);
}
void printRefund() {
  System.out.println("Shop");
  System.out.println(date);
  System.out.println(-total);
}
```

```
private void printHeader() {
  System.out.println("Shop");
  System.out.println(date);
}
void printReceipt() {
  printHeader();
  System.out.println(total);
}
void printRefund() {
  printHeader();
  System.out.println(-total);
}
```

Your task: Pull the three repeated print lines into one private method, and pull the charge calculation out into `calculateCharge()` so printing and calculating are separate.

> **Hint**
> `Math.min(a, b)` returns the smaller value, so it can replace an `if (c > 1000) c = 1000;` cap in one line.

B5 · Replace Type Code with Subclasses (Replace Conditional with Polymorphism)

Fixes: type code `t` with the same `if (t == 1) ... else if (t == 2)` repeated in two methods

Small example (not the parking code). Before → after:

```
double area() {
  if (type == 1) return w * h;
  else if (type == 2)
    return 3.14 * r * r;
}
```

```
abstract class Shape {
  abstract double area();
}
class Rect extends Shape {
  double area() { return w * h; }
}
class Circle extends Shape {
  double area() { return 3.14*r*r; }
}
```

Your task: Create an abstract parent class and two subclasses, `Car` and `Bike`. Each subclass answers two questions: what is my rate per hour, and what is my type name?

> **Hint**
> If the parent asks `getRatePerHour()` and each child returns its own value, every `if (t == ...)` disappears.

B6 · Remove Dead Code

Fixes: unused method `oldDisplay()` and unused field `z`

Small example (not the parking code). Before → after:

```
int legacyTotal;   // unused
void oldSave() { ... } // never called
```

```
// deleted: Git history keeps
// the old version if you
// ever need it
```

Your task: Prove nothing uses them (Section E will help), then delete them.

> **Hint**
> Dead code still costs: every reader has to work out whether it matters.

Write your Version 2.0 here (also run it and attach the console output)

Version 2.0: Vehicle (abstract), Car, Bike, main

Check: does your output match the expected output on page 2, line for line? ☐ Yes ☐ Not yet

C Smell-detection tools

Tools automate Section A. Match each tool to its description by writing the letter.

Tool	Letter	Description
1. PMD		(a) Eclipse plug-in for Java that finds design smells such as God Class, Feature Envy, Long Method and type checking (switch on a type code), and proposes the refactoring to fix each one.
2. Checkstyle		(b) Java static analyser with rules for unused private variables and methods, empty catch blocks and overly long methods. Comes with CPD, a copy-paste (duplicate code) detector.
3. SonarQube		(c) Server platform that scans many languages (Java, C#, Python, JavaScript and more) for code smells, bugs, duplication and cognitive complexity, and shows them on a dashboard.
4. JDeodorant		(d) Enforces a coding standard: naming conventions, magic numbers, method length, missing Javadoc.

C.1 Which tool would flag line 1 (`class vehicle`) and line 12 (`1000`)? _____

C.2 Which smell from Section A would JDeodorant be best at finding? _____

C.3 PMD's 'unused private method' rule would *not* flag `oldDisplay()`. Why not?

Hint
Look at the access modifier on line 31. A tool can only be sure a method is unused if nothing outside the class could call it.

D Benefits check: apply a change request

The parking manager sends three change requests. Count how many **existing** places you must edit in each version.

Change request	Edits in Version 1.0	Edits in your Version 2.0
CR-1: Car rate becomes Rs. 60 per hour		
CR-2: Maximum charge becomes Rs. 1500		
CR-3: Add a new vehicle type, Truck, at Rs. 100 per hour		

D.1 From your table, write **three** benefits of the refactoring you did:

Hint
Think about: reading it (readability), changing a rule in one place (maintainability), adding a type without touching tested code (extensibility, Open/Closed principle), and fewer places for a bug to hide.

E Program comprehension: chunking and cross-referencing

Two ideas in two lines
Chunking: grouping lines you recognise as one idea and naming that group, so you read 5 ideas instead of 20 lines. Chunks build up: lines → method → class → module.
Cross-referencing: linking a name's definition to every place it is used (IDE: Ctrl+Click, Find Usages, Call Hierarchy; or grep), so you can see how parts of a program connect.

E1 · Chunking

This method from the parking system has no comments. Draw a bracket around each group of lines that does one job, and label each bracket in three words or fewer. Aim for 5 chunks.

```
1  void exitParking(String plate, String slotId) {
2      if (plate == null || slotId == null) {
3          System.out.println("Invalid input");
4          return;
5      }
6      Slot slot = lot.find(slotId);
7      Vehicle vehicle = registry.find(plate);
8      slot.setFree(true);
9      slot.setVehicle(null);
10     int charge = vehicle.calculateCharge();
11     if (charge > 0) {
12         wallet.deduct(plate, charge);
13         sms.send(vehicle.getPhone(), "Paid Rs. " + charge);
14     }
15     log.write(plate + " left slot " + slotId);
16 }
```

Chunk labels

Hint
Blank lines are missing on purpose. Look for where the verb changes: check, find, update, charge, record.

E2 · Cross-referencing

Go back to the **Version 1.0** code on page 2 and fill in the table.

Name	Defined at line	Used at line(s)
`t`		
`h`		
`process()`		
`oldDisplay()`		
the number `1000`	(not defined: it's a literal)	

E2.1 What does your table prove about `oldDisplay()`? **E2.2** Why was searching for h painful in a text search?

E3 · Put them together

Imagine the parking system has 200 files. In two sentences, explain how you would use chunking *and* cross-referencing together to find and understand where parking charges are calculated.

✓ Reflection

One smell I will now always notice: _____ One refactoring I can now apply: _____

Submission instructions

- Submit **one PDF per pair** by **Monday, 26 October 2026**.
- Name the file `SREE_A1_G1_RollNo1_RollNo2.pdf`.
- Include your completed answers, your Version 2.0 Java code, and a screenshot of its console output.
- Plagiarism policy (course outline): all parties involved in the first cheating case are awarded zero for this evaluation.